Лабораторная работа 3 "Реализация системы аутентификации и авторизации"

Лабораторная работа №3. Реализация системы аутентификации и авторизации

Ветка репозитория: `23`

---

1. ЦЕЛЬ РАБОТЫ

Освоение механизмов аутентификации и авторизации в веб-приложениях. Реализация регистрации и входа пользователей с использованием JWT (JSON Web Tokens), хеширование паролей с помощью bcrypt, защита маршрутов с помощью middleware **для продолжения разработки серверной части вашего курсового проекта**.

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- Понятия аутентификации и авторизации.
- Принцип работы JWT (структура токена, подпись, срок действия).
- Хеширование паролей (bcrypt, соль).
- Middleware в Express.js для проверки токена.
- Безопасное хранение токенов на клиенте (localStorage / cookies).

---

3. ЗАДАЧИ РАБОТЫ

- Создать модель пользователя (email, хеш пароля). Рекомендуется добавить модель в существующий проект, используя ту же базу данных (PostgreSQL) и ORM Sequelize из лабораторной работы №2.
- Реализовать маршрут регистрации (хеширование пароля, сохранение в БД).
- Реализовать маршрут входа (проверка пароля, выдача JWT).
- Создать middleware для проверки JWT.

- Защитить некоторые маршруты (только для авторизованных пользователей).
- Реализовать маршрут для получения информации о текущем пользователе.
- **В соответствии с вашей Темой курсового проекта (определяется Генератором вариантов.) добавить один из дополнительных механизмов безопасности (см. раздел 7).**

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ (общий для всех вариантов)

Часть 1. Установка зависимостей

```
npm install jsonwebtoken bcrypt
```

(При использовании переменных окружения установите также `dotenv`.)

Часть 2. Модель пользователя (Sequelize)

Создайте модель `User`:

```
npx sequelize-cli model:generate --name User --attributes email:string,passwordHash:string
```

Обязательные поля:

- `id` (первичный ключ)
- `email` (string, unique, allowNull: false)
- `passwordHash` (string, allowNull: false)

При необходимости добавьте дополнительные поля в зависимости от выбранного дополнительного механизма (например, `role`, `refreshToken`, `failedLoginAttempts`, `lockUntil`).

Выполните миграцию:

```
npx sequelize-cli db:migrate
```

Часть 3. Маршрут регистрации (POST /auth/register)

Создайте файл `routes/auth.js`. Реализуйте эндпоинт:

- Принимает `email` и `password`.

- Проверяет уникальность email.
- Хеширует пароль: `bcrypt.hash(password, 10)`.
- Сохраняет пользователя.
- Возвращает статус 201.

Часть 4. Маршрут входа (POST /auth/login)

- Находит пользователя по email.
- Сравнивает пароль с хешем.
- Генерирует JWT: `jwt.sign({ id, email }, process.env.JWT_SECRET, { expiresIn: '1h' })`.
- Возвращает токен.

Часть 5. Middleware аутентификации

Создайте `middleware/auth.js`:

- Извлекает токен из заголовка `Authorization: Bearer <token>`.
- Верифицирует токен.
- Прикрепляет `req.user = decoded` и вызывает `next()`.

Часть 6. Защищённые маршруты

- Защитите маршрут `GET /profile` (возвращает данные текущего пользователя).
- По желанию добавьте другие защищённые маршруты (например, удаление пользователя).

Часть 7. Реализация дополнительного механизма (по теме курсового проекта)

В зависимости от специфики вашего курсового проекта выберите **один** из предложенных ниже механизмов или согласуйте с преподавателем свой.

Механизм	Краткое описание
Ролевая модель (RBAC)	Добавить поле `role` (`user`, `admin`). Создать middleware `isAdmin`. Защитить маршруты, доступные только администраторам.

Механизм	Краткое описание
Refresh-токены	Добавить поле `refreshToken`. Создать эндпоинт `POST /auth/refresh` для выдачи новой пары токенов.
Блокировка после неудачных попыток входа	Поля `failedLoginAttempts`, `lockUntil`. После 5 неудачных попыток блокировать учётную запись на 5 минут.
Смена пароля	Эндпоинт `POST /auth/change-password` с проверкой старого пароля и хешированием нового.
Логирование входов	Модель `LoginLog` (userId, ip, userAgent, timestamp, success). Эндпоинт для просмотра логов администратором.
Двухфакторная аутентификация (email)	Генерация одноразового кода, отправка на email (имитация – вывод в консоль), проверка на отдельном эндпоинте.
Проверка сложности пароля	При регистрации и смене пароля проверять длину, наличие заглавных букв, цифр, спецсимволов.
Подтверждение email	Генерация токена, отправка ссылки на email (имитация), поле `isVerified`, запрет входа без подтверждения.
Ограничение активных сессий	Модель `Session` с `tokenVersion`, запрет более 3 одновременных сессий.
Восстановление пароля через email	Эндпоинты `forgot-password` (генерация токена) и `reset-password` (смена пароля по токену).
«Запомнить меня»	Параметр `rememberMe` при логине, увеличение времени жизни токена.
Блокировка пользователей администратором	Поля `isBanned`, `banReason`, middleware `isNotBanned`, эндпоинты `/ban` и `/unban`.
Имитация OAuth (GitHub)	Создать заглушку входа через GitHub, возвращающую JWT.
Защита от CSRF	Генерация CSRF-токена, проверка в заголовке `X-CSRF-Token` (кроме GET и auth-эндпоинтов).
Смена email	Эндпоинт `POST /auth/change-email` с проверкой пароля и повторным подтверждением по email.
SMS-подтверждение	Генерация 6-значного кода, вывод в консоль, проверка на отдельном эндпоинте.

Примечание: Выбор конкретного механизма может быть подсказан предметной областью вашего курсового проекта (например, для интернет-магазина важны роли, для системы задач – восстановление пароля). При необходимости согласуйте выбор с преподавателем.

---

5. СОДЕРЖАНИЕ ОТЧЕТА

- Цель работы, краткие теоретические сведения о JWT.
- Листинг модели пользователя и middleware аутентификации.
- Скриншоты успешной регистрации, входа с получением токена.
- Скриншот запроса к защищённому маршруту с валидным токеном.
- Скриншот запроса к защищённому маршруту без токена (401).
- Описание и скриншоты реализации дополнительного механизма.
- Ответы на контрольные вопросы.
- Выводы.

---

6. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.
- Ссылка на репозиторий GitHub с кодом (ветка 23).
- **Индивидуальное выполнение. Вариант** **Темы** определяется по **Генератору вариантов.**
- Дополнительный механизм выбирается в соответствии с темой проекта (или согласовывается с преподавателем).

---

7. КОНТРОЛЬНЫЕ ВОПРОСЫ (16 вопросов)

1. В чём разница между аутентификацией и авторизацией?
2. Из каких частей состоит JWT? Как проверить его подлинность на сервере?
3. Почему пароли нужно хешировать? Что такое соль (salt)?
4. Где безопаснее хранить JWT на клиенте (localStorage или httpOnly cookie)? Почему?
5. Как защитить маршрут, доступный только администратору?

6. Как реализовать выход пользователя (logout) при использовании JWT?
7. Какие недостатки имеет хранение JWT в localStorage (уязвимость XSS)?
8. Как работает механизм обновления токенов (refresh token)?
9. Как ограничить количество одновременно действующих сессий для одного пользователя?
10. Как защититься от атак по подбору пароля (brute force) на уровне API?
11. Что произойдёт, если скомпрометирован секретный ключ JWT?
12. Как реализовать временную блокировку учётной записи после неудачных попыток входа?
13. В каких заголовках передаётся JWT обычно? Какой формат заголовка?
14. Как добавить время жизни токена и обеспечить его автоматическое истечение?
15. Можно ли отозвать JWT до истечения его срока? Какие есть подходы?
16. Сравните сессионную аутентификацию (с куками) и JWT. Когда что лучше?

---

8. ПРИМЕЧАНИЯ

- Работа выполняется **индивидуально**.
- Логически продолжает лабораторные работы №1 и №2 (база данных уже должна быть подключена, созданы модели вашей предметной области).
- Все используемые инструменты (Node.js, Express, PostgreSQL, Sequelize, bcrypt, jsonwebtoken) являются локальными или свободно распространяемыми. Облачные сервисы (Neon, Supabase) могут использоваться, но не обязательны.
- В следующей лабораторной работе (№4) вы интегрируете React-приложение с этим API, используя JWT для авторизации запросов.

---

9. КРИТЕРИИ ОЦЕНКИ

- Работоспособность регистрации и входа.
- Корректная генерация и проверка JWT.
- Реализация middleware и защита маршрута.

- Выполнение дополнительного механизма безопасности (в соответствии с темой курсового проекта).
- Оформление отчёта и ответы на контрольные вопросы.
- Ссылка на репозиторий с веткой `23`.